

CSE145 – Introduction to Computing

Assignment 2: Turing Machines — Design, Analysis & Simulation

Name: Hareem Saeed

ERP No.: 36639

Personalization block: R = 39, V = 3, B = 100111, n = 5

Github Repository Link:

<https://github.com/hareemsaeed/ITC.git>

Question 1:

Task A:

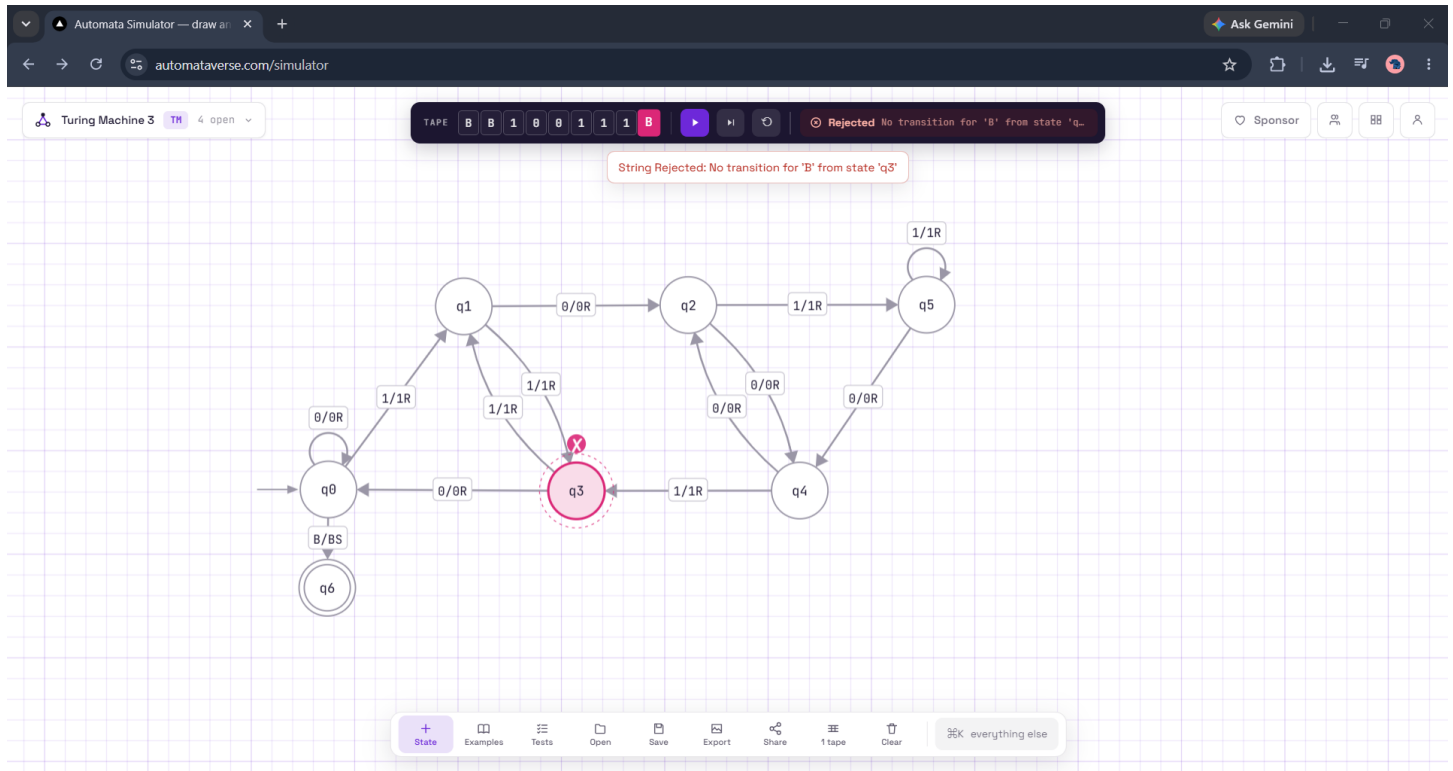
My $M = 6$ for this task. Every number is either divisible by 6 (remainder 0) or gives a remainder of 1, 2, 3, 4, or 5 when divided by 6. Each state represents the remainder when divided by 6. For example, q_0 means remainder = 0, q_1 means remainder = 1, q_2 means remainder = 2, and so on. Every time a 0 is added to the end of a binary string, it multiplies the value of the binary sequence by 2. If a 1 is added to the end, the value gets multiplied by 2 and 1 is added. The value that needs to be remembered as every bit is added is the previous state or remainder, and we get a general formula: $[2(\text{remainder}) + \text{current bit}] \bmod 6$. Doing this step at every state gives us the answer for the next move.

Task B:

Current State	Read	Write	Move	Current Remainder	$(2R+\text{bit})$	New Remainder (MOD 6)	Next State
q_0	0	0	R	0	$2(0)+0=0$	0	q_0
q_0	1	1	R	0	$2(0)+1=1$	1	q_1
q_1	0	0	R	1	$2(1)+0=2$	2	q_2
q_1	1	1	R	1	$2(1)+1=3$	3	q_3
q_2	0	0	R	2	$2(2)+0=4$	4	q_4
q_2	1	1	R	2	$2(2)+1=5$	5	q_5
q_3	0	0	R	3	$2(3)+0=6$	0	q_0
q_3	1	1	R	3	$2(3)+1=7$	1	q_1
q_4	0	0	R	4	$2(4)+0=8$	2	q_2
q_4	1	1	R	4	$2(4)+1=9$	3	q_3
q_5	0	0	R	5	$2(5)+0=10$	4	q_4
q_5	1	1	R	5	$2(5)+1=11$	5	q_5
q_0	B	B	S	0	N/A	N/A	$q_6(\text{Accept State})$

Task C:

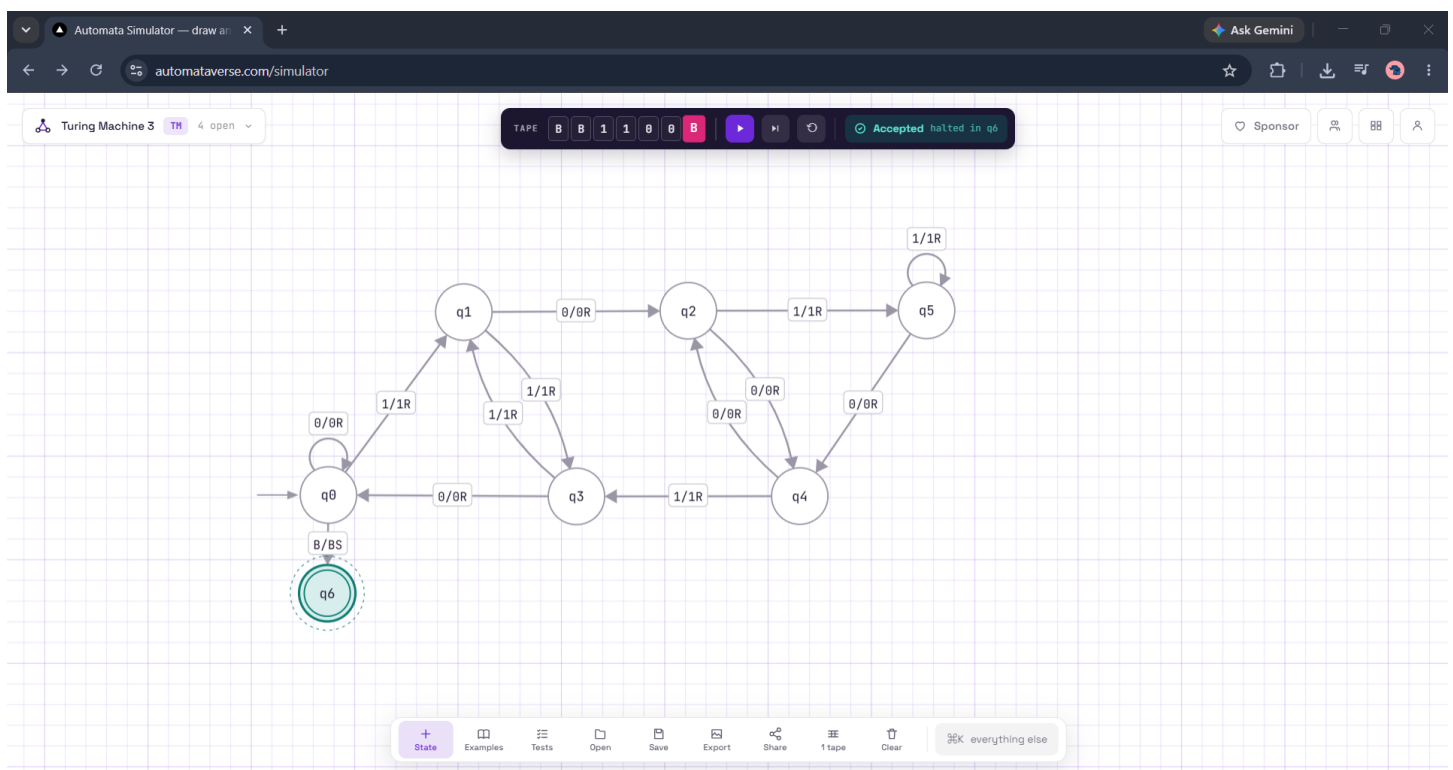
i) Run 1: input 100111 - <https://www.automataverse.com/m/fM7xeBBW>



Status: Rejected (Correct, because 39 is not divisible by 6.)

GitHub evidence: Q1_run1.png and Q1_run1_file.json

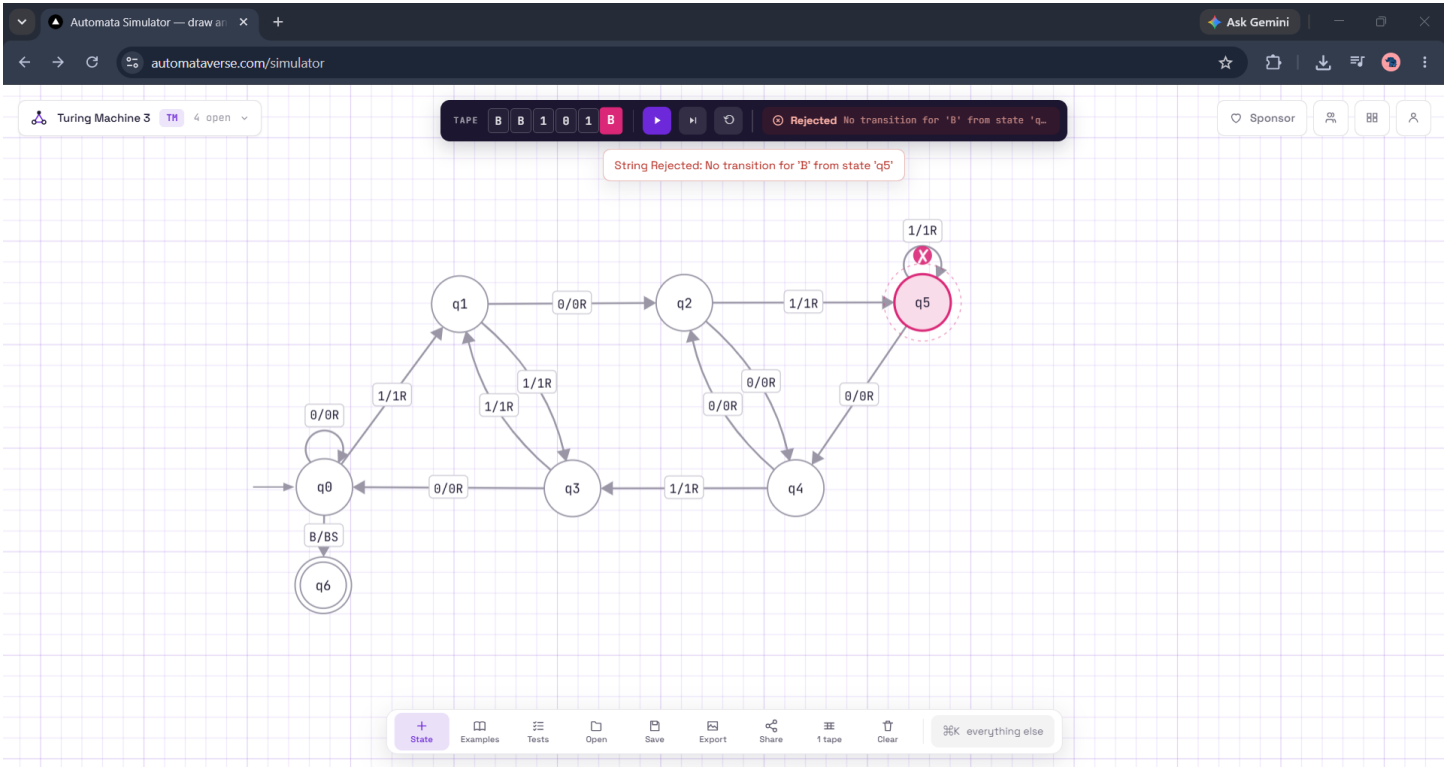
ii) Run 2: input 1100 - <https://www.automataverse.com/m/V2W3tvYm>



Status: Accepted (Correct, because 12 is divisible by 6.)

GitHub evidence: Q1_run2.png and Q1_run2_file.json

iii) Run 3: input 101 - <https://www.automataverse.com/m/xe5urQfu>



Status: Rejected (Correct, because 5 is not divisible by 6.)

GitHub evidence: Q1_run3.png and Q1_run3_file.json

Task D:

A machine built for one value of M cannot be reused unmodified for a different value of M because, in this case, the TM is made keeping in mind all the remainders that can possibly be an answer when dividing by M (which is 6). For other values of M, either fewer or more states may be needed. For example, if M was 5, then q5 shouldn't exist because there can't be a remainder of 5. Similarly, if M was 7, an input of binary 6 would lead to acceptance even when it isn't divisible by 7, and another state, q6, would need to be added, which would not be an accepting state.

Question 2:

Task A:

I have used the strategy of crossing out one a, followed by one b and one c. I move from the start to the end of the sequence and then move left to come back to the start again. I repeat this process until there are no more unmarked a's left. Once all my a's have been replaced by X, I take a different path and check if the rest of the string only contains Y and Z. If it does, the machine accepts the string.

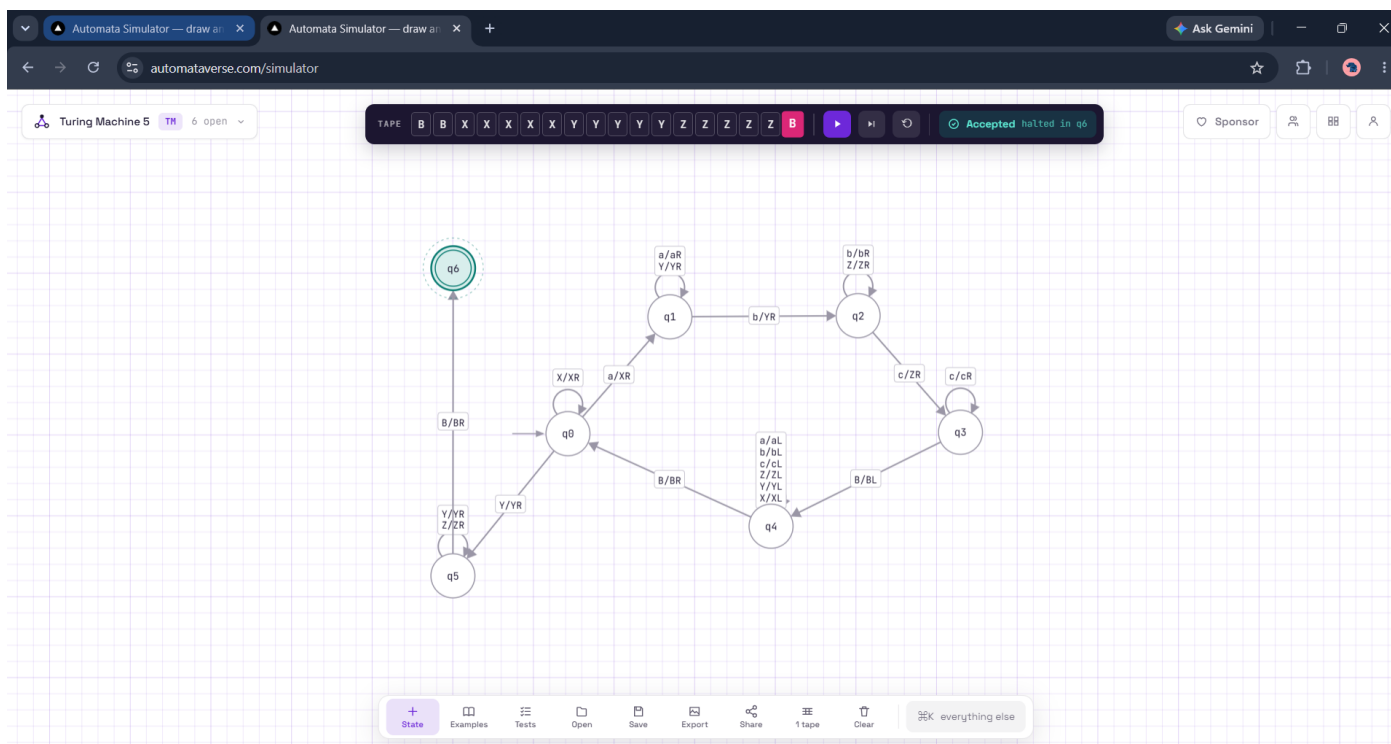
Task B:

Current State	Read	Write	Move	Next State
q0	X	X	R	q0
q0	a	X	R	q1
q0	Y	Y	R	q5

q1	a	a	R	q1
q1	Y	Y	R	q1
q1	b	Y	R	q2
q2	b	b	R	q2
q2	Z	Z	R	q2
q2	c	Z	R	q3
q3	c	c	R	q3
q3	B	B	L	q4
q4	a	a	L	q4
q4	b	b	L	q4
q4	c	c	L	q4
q4	X	X	L	q4
q4	Y	Y	L	q4
q4	Z	Z	L	q4
q4	B	B	R	q0
q5	Y	Y	R	q5
q5	Z	Z	R	q5
q5	B	B	R	q6

Task C:

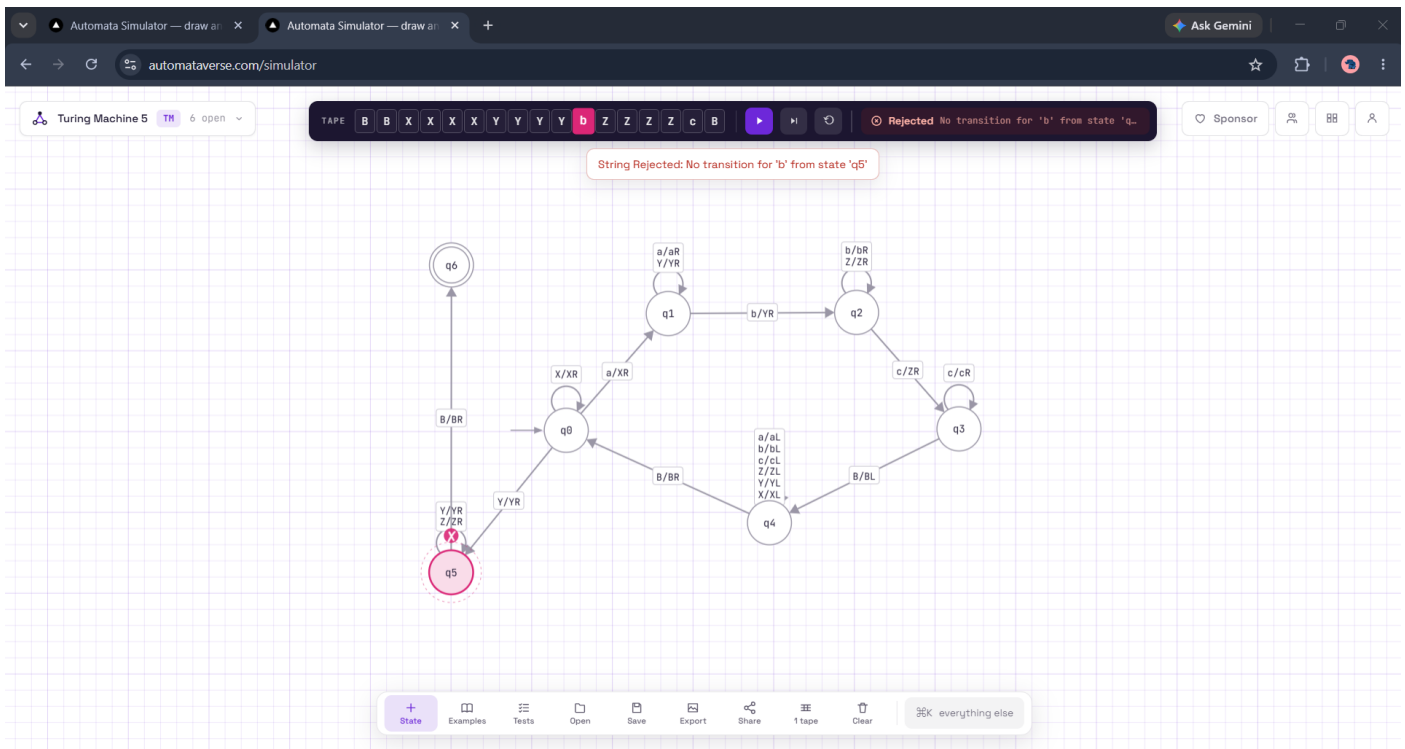
- i) Run 1: aaaaabbbbcccc (for n=5) – <https://www.automataverse.com/m/vtHzC4oF>



Status: Accepted

GitHub evidence: Q2_run1.png and Q2_TM_file.json

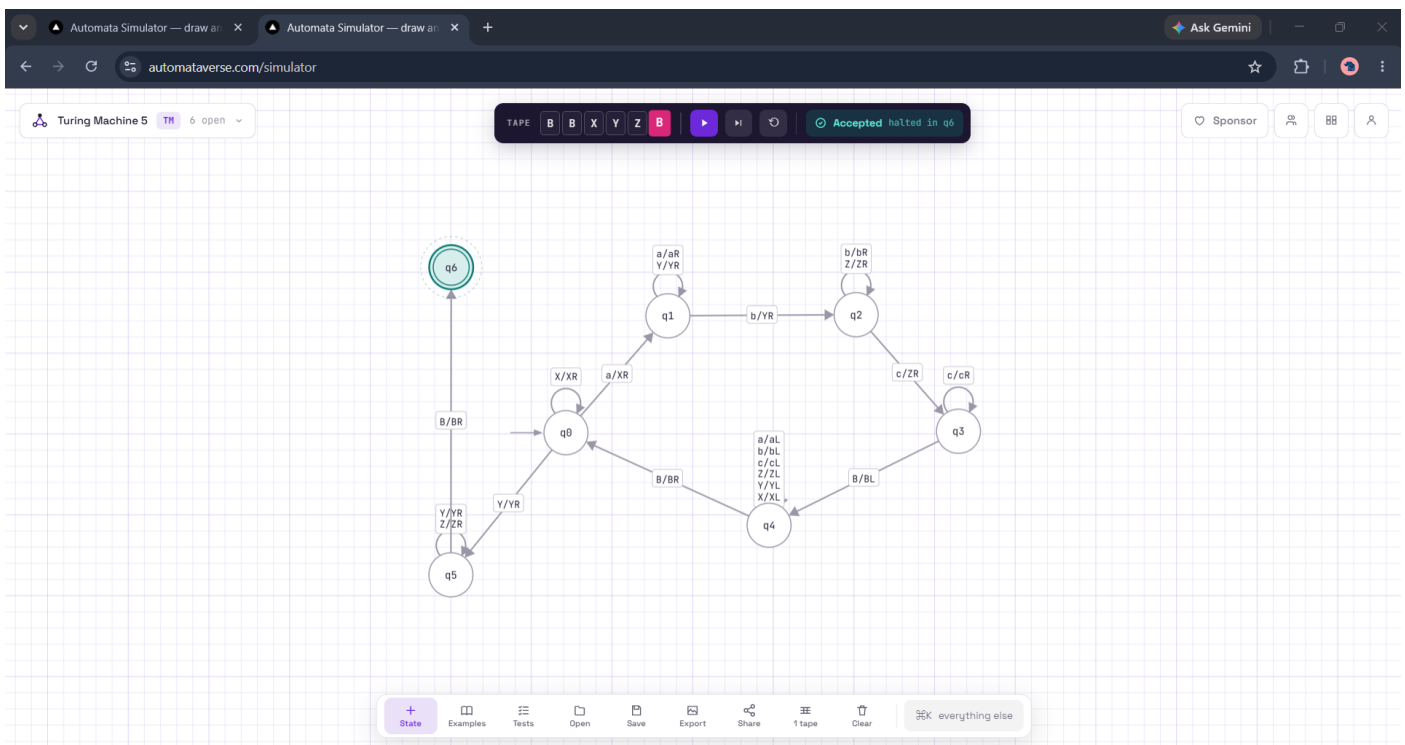
- ii) Run 2: aaaabbbbcccc (1 a missing) – <https://www.automataverse.com/m/vtHzC4oF>



Status: Rejected

GitHub evidence: Q2_run2.png and Q2_TM_file.json

iii) Run 3: abc - <https://www.automataverse.com/m/vtHzC4oF>



Status: Accepted

GitHub evidence: Q2_run3.png and Q2_TM_file.json

Task D:

The most likely place for an off-by-one error in my machine would be when I return to q_0 to look for the next unmarked a or during the final check of the Y's and Z's. I could accidentally skip one symbol or stop before checking the last symbol. I convinced myself that my machine does not have this error by testing a string with equal numbers of a's, b's and c's and a string with one symbol missing. For example, `aaaaabbbbcccc` should be accepted, while `aaaaabbbbcccc` should be rejected. This helped me check that every a is matched with exactly one b and one c and that the final symbols are also checked.

Question 3:

Task A:

Every time a binary number undergoes integer division by 2, its last bit gets removed. For example, $11_2 = 7$ and $7/2 = 3.5$, which becomes 3. If I were to remove the last bit, the value would become $11_2 = 3$. This case, however, is different for 0 and 1. When $1/2$ is 0, if we remove the only bit we have, the answer becomes blank, which is wrong. Similarly, if we have only 0, then $0/2 = 0$; however, if we remove the digit, it becomes blank and not 0. After this thought process, I added 2 different conditions for what to do if the input is only 0 or only 1.

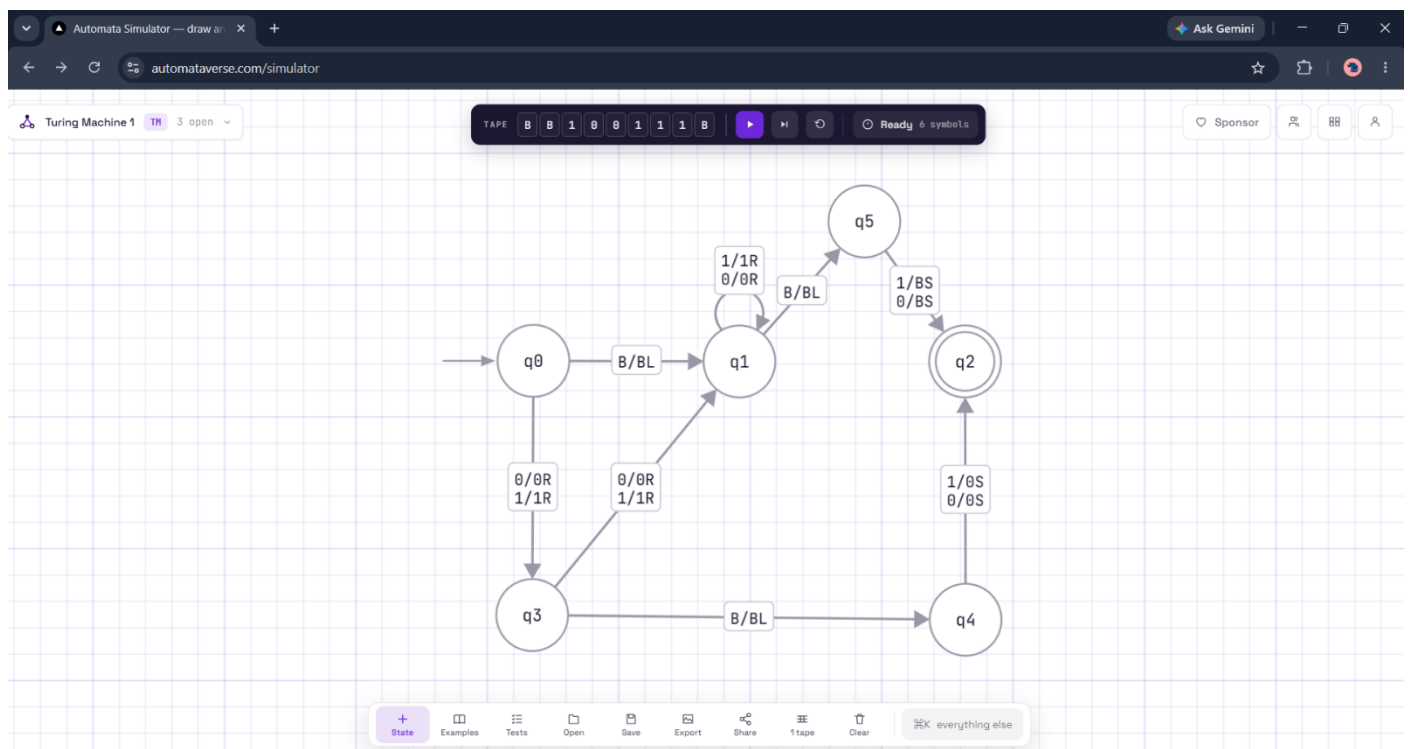
Task B:

Current State	Read	Write	Move	Next State	Purpose
q_0	0	0	R	q_3	Move right until a blank is reached
q_0	1	1	R	q_3	Move right until a blank is reached
q_0	B	B	L	q_1	Move left after reaching the blank to find the last bit
q_3	0	0	R	q_1	Check whether this is the only digit
q_3	1	1	R	q_1	Check whether this is the only digit
q_3	B	B	L	q_4	Handle the special case of a single-digit input
q_4	0	0	S	q_2	Handle $0/2 = 0$ and halt
q_4	1	0	R	q_2	Handle $0/2 = 0$ and halt
q_1	0	0	R	q_1	Move right until a blank is reached
q_1	1	1	R	q_1	Move right until a blank is reached
q_1	B	B	L	q_5	Move left after reaching the blank to find the last bit
q_5	0	B	R	q_2	Delete the last bit to perform division by 2
q_5	1	B	R	q_2	Delete the last bit to perform division by 2

Task C:

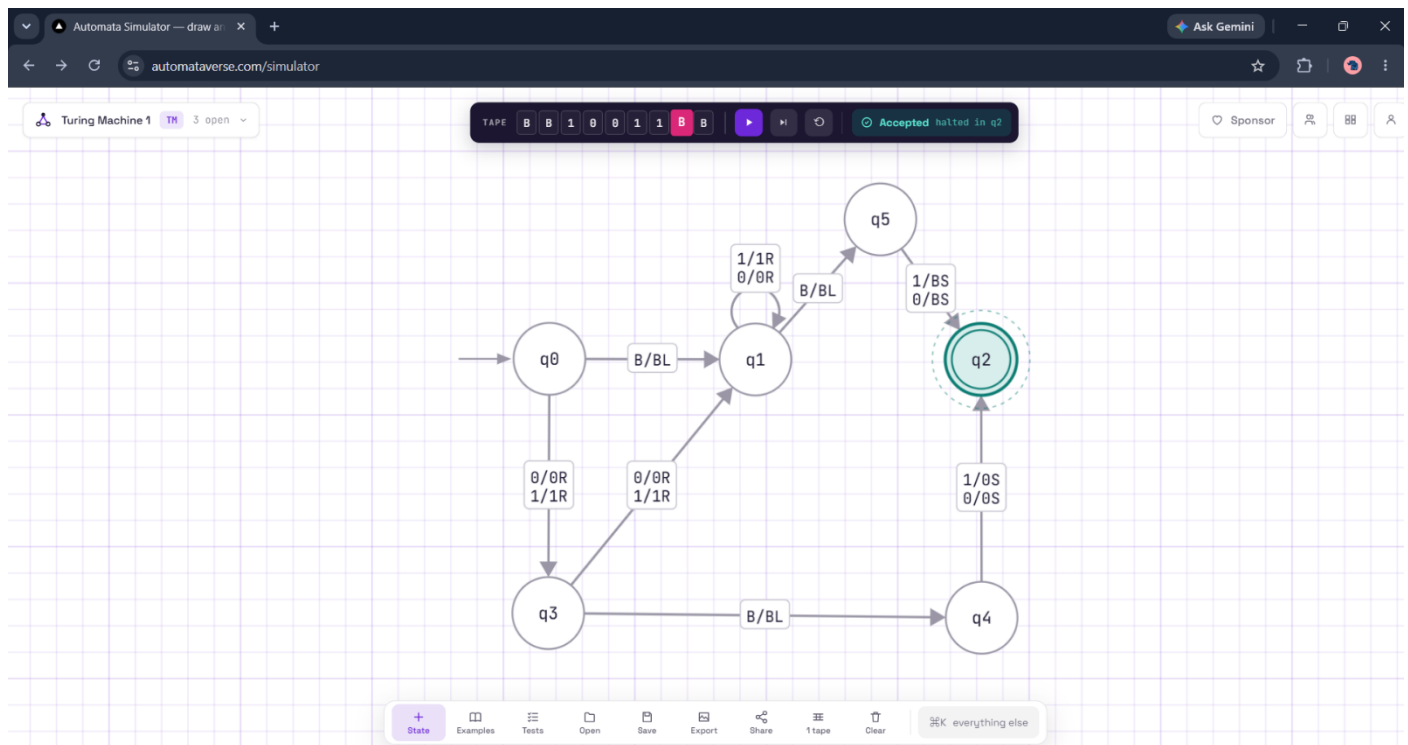
Run 1: 100111- <https://www.automataverse.com/m/cBZa2VtG>

Before:



GitHub evidence: Q3_run1_initial.png and Q3_run1_file.json

Final:



GitHub evidence: Q3_run1_final.png and Q3_run1_file.json

Verification:

$$100111_2 = 39$$

$$39/2 = 19.5 \sim 19$$

$$19 = 10011_2$$

Therefore, the final tape content **10011** matches the expected result.

Task D:

The two edge cases I have identified in this question are when we have only one bit, which is 0 or 1, in the input. Removing the bit does not give the correct answer because it gives a blank, which is no value at all. Therefore, to tackle this problem, I applied special conditions that handle $0/2 = 0$ and $1/2 = 0$.

Question 4:

Task A:

The strategy I used is to compare the symbols from the outside going inwards. Since this is an anti-palindrome, the two symbols at matching positions from the ends must be different. So, if the left symbol is 0, the right symbol must be 1, and if the left symbol is 1, the right symbol must be 0. The Turing Machine first finds the leftmost unmarked symbol and marks it. It remembers whether it was a 0 or a 1 in its state, then moves to the right end of the string. From there, it finds the rightmost unmarked symbol and checks whether it is the opposite symbol. If it is correct, it marks it and moves back to the left to repeat the same process for the next pair. This continues until all the symbols have been checked. If a pair contains the same symbols, the string is rejected. If all the pairs contain opposite symbols, the string is accepted. This also means that a string with an odd number of symbols is rejected because there will be one symbol left in the middle without a matching symbol.

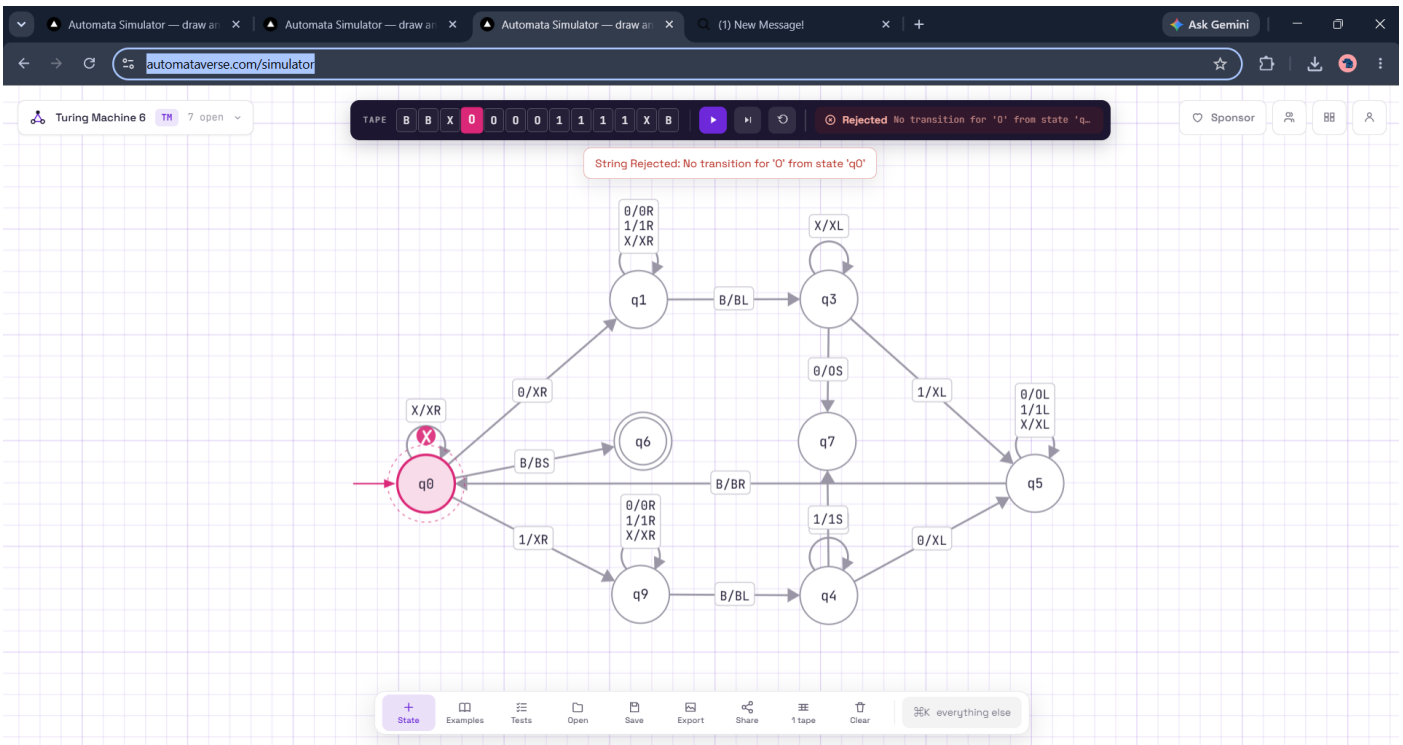
Task B:

State	Read	Write	Move	Next State
q0	X	X	R	q0
q0	0	X	R	q1
q0	1	X	R	q9
q0	B	B	R	q6
q1	0	0	R	q1
q1	1	1	R	q1
q1	X	X	R	q1
q1	B	B	L	q3
q3	X	X	L	q3
q3	1	X	L	q5
q3	0	0	R	q7
q3	B	B	R	q7
q9	0	0	R	q9
q9	1	1	R	q9
q9	X	X	R	q9

q9	B	B	L	q4
q4	X	X	L	q4
q4	0	X	L	q5
q4	1	1	R	q7
q4	B	B	R	q7
q5	0	0	L	q5
q5	1	1	L	q5
q5	X	X	L	q5
q5	B	B	R	q0

Task C:

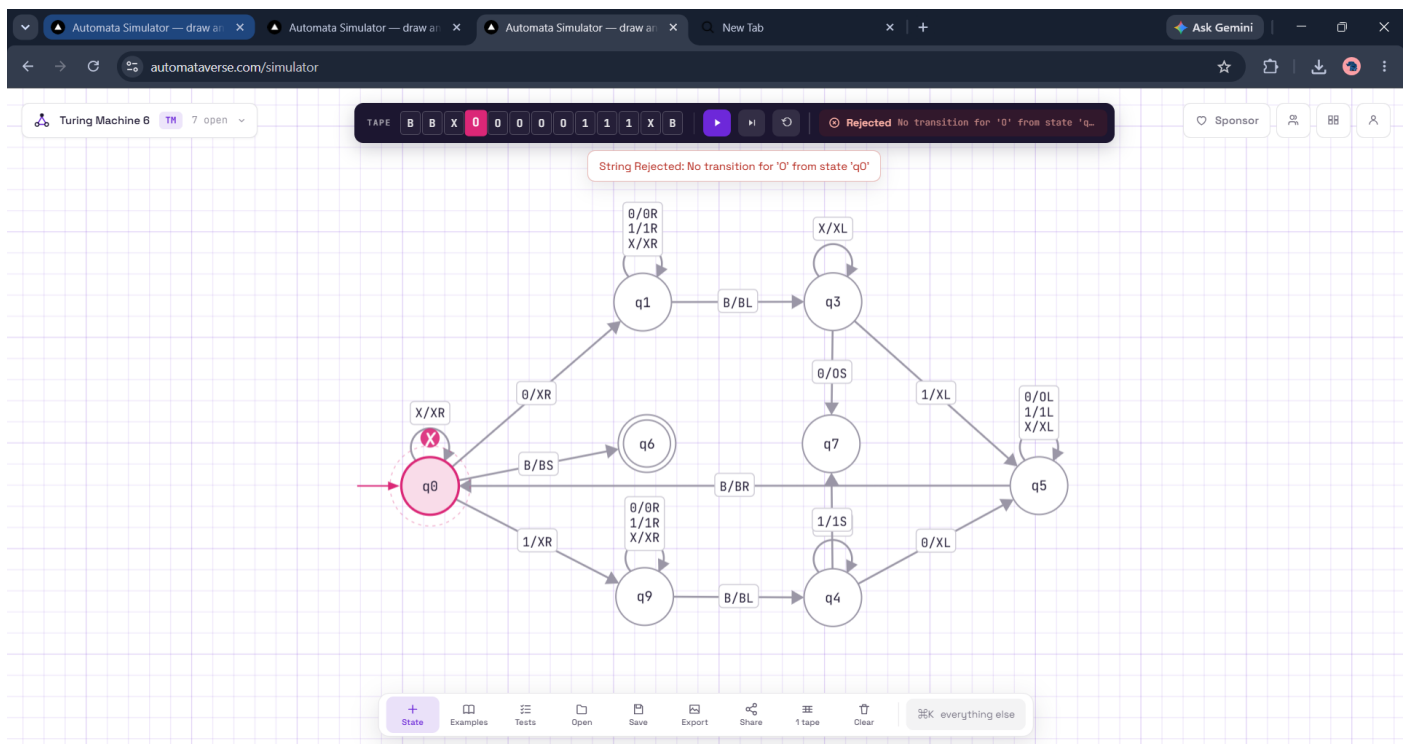
i) Run 1: 0000011111 - <https://www.automataverse.com/m/Ev5tjHG2>



Status: Rejected (This sequence is meant to be **ACCEPTED**. However, my Turing Machine was acting up during the run. As you can see above, it says that there is no transition for 0, even though there is clearly a transition for 0 in the machine. I tried fixing the problem multiple times, including recreating the transition, but it still would not work.)

GitHub evidence: Q4_run1.png and Q4_TM_file.json

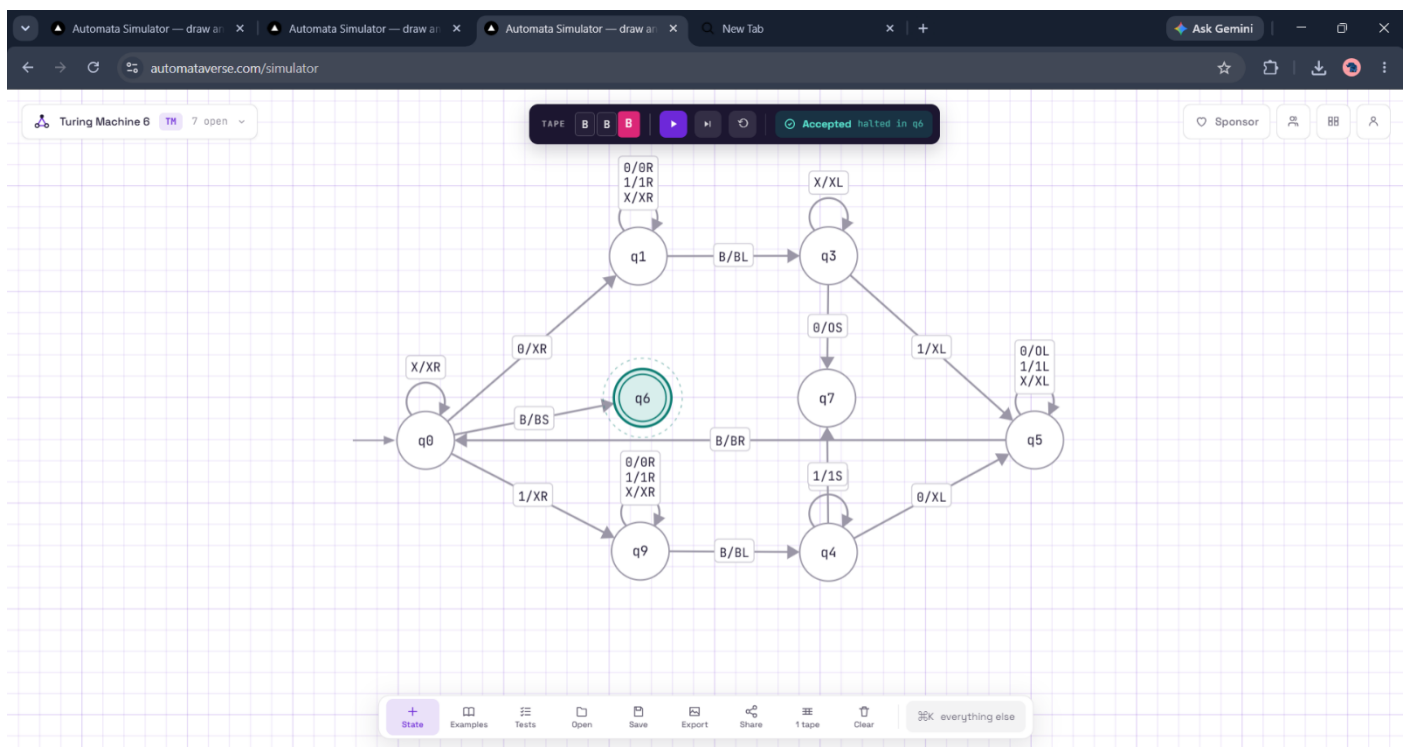
ii) Run 2: 0000001111 - <https://www.automataverse.com/m/Ev5tjHG2>



Status: Rejected (This sequence is meant to be rejected. However, my Turing Machine was acting up during the run. As you can see above, it says that there is no transition for 0, even though there is clearly a transition for 0 in the machine. I tried fixing the problem multiple times, including recreating the transition, but it still would not work.)

GitHub evidence: Q4_run2.png and Q4_TM_file.json

iii) Run 3: “ ” - <https://www.automataverse.com/m/Ev5tjHG2>



Status: Accepted

GitHub evidence: Q4_run3.png and Q4_TM_file.json

Task D:

Multiple left-to-right and right-to-left sweeps are needed because the Turing Machine only has one tape head. It cannot look at the first and last symbols at the same time. The machine first reads a symbol from the left and remembers it in its state. It then has to move all the way to the right to find the opposite symbol at the other end. After checking and marking that symbol, it has to move back to the left again to find the next unmarked symbol. This process is repeated for every pair, moving further towards the middle each time. Therefore, one left-to-right pass is not enough because the machine needs to compare symbols from opposite ends of the string.

Question 5:

Task A:

My strategy is to compare the as on the left side of # with the bs on the right side. I start by finding the first a that has not been marked yet and mark it as X. Then I move to the right, cross the #, and look for an unmarked b. When I find one, I mark it as Y and move all the way back to the left side to find the next unmarked a. I keep repeating this process until there are no unmarked as left. After that, I check the right side to see if there are any unmarked bs remaining. If there are no bs left, it means every a had a matching b, so the string is accepted. If there is a b left over, or if I cannot find a b to match an a, the string is rejected.

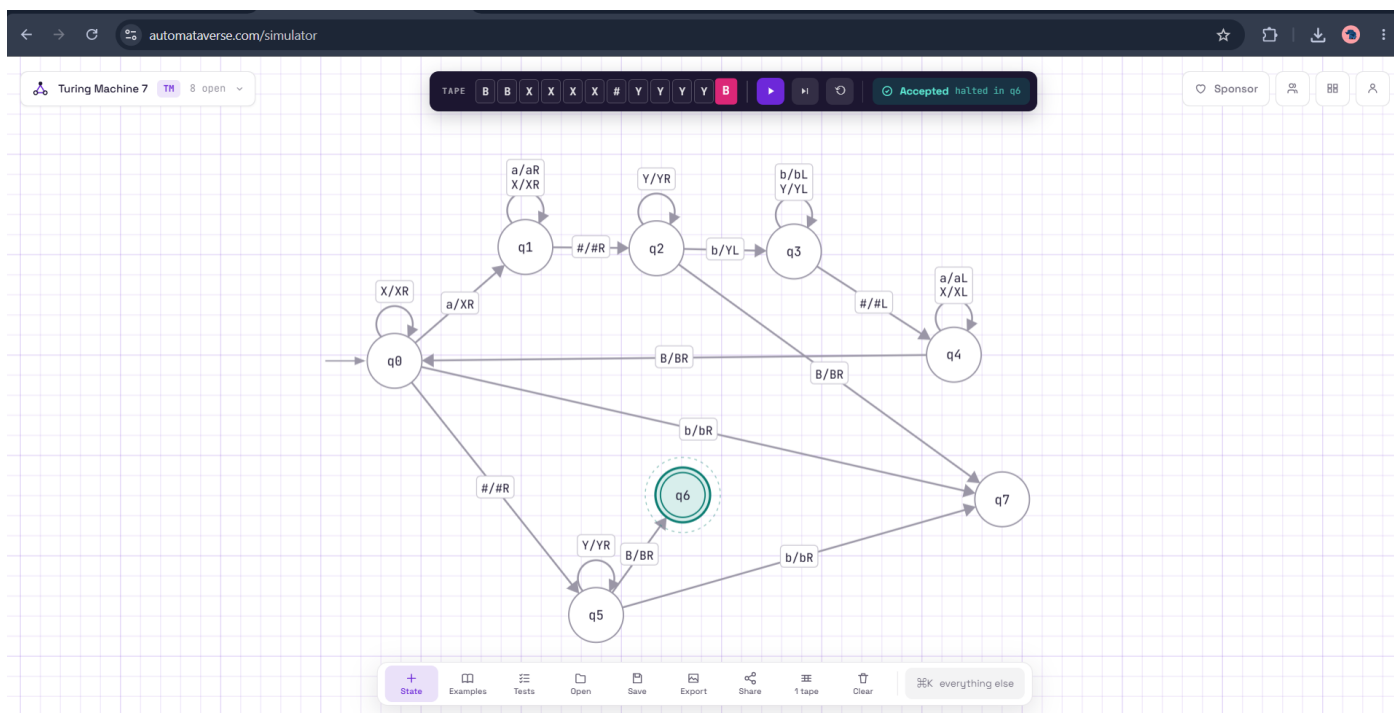
Task B:

State	Read	Write	Move	Next
q0	a	X	R	q1
q0	X	X	R	q0
q0	#	#	R	q5
q0	b	b	R	q7
q1	a	a	R	q1
q1	X	X	R	q1
q1	#	#	R	q2
q2	Y	Y	R	q2
q2	b	Y	L	q3
q2	B	B	R	q7
q3	b	b	L	q3
q3	Y	Y	L	q3
q3	#	#	L	q4
q4	a	a	L	q4

q4	X	X	L	q4
q4	B	B	R	q0
q5	Y	Y	R	q5
q5	b	b	R	q7
q5	B	B	R	q6

Task C:

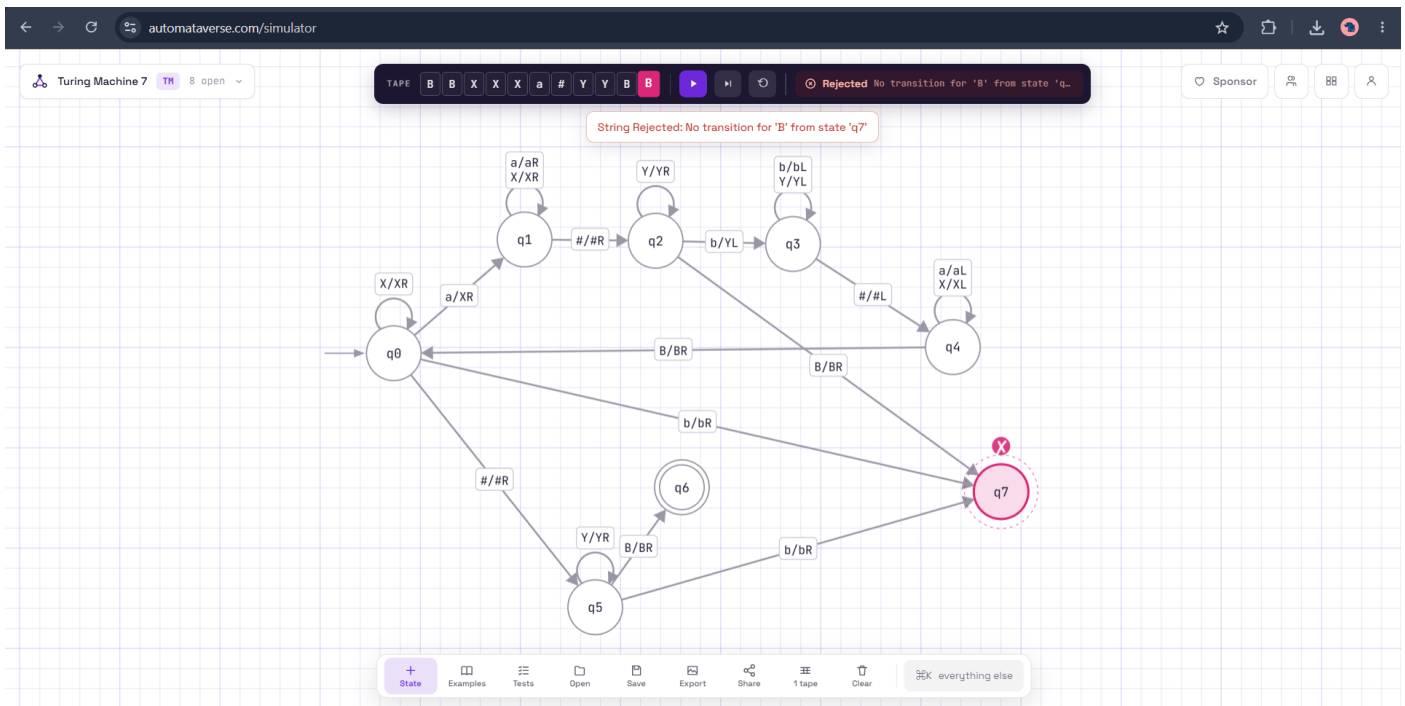
i) Run 1: aaaa#bbbb - <https://www.automataverse.com/m/R7BLRYbV>



Status: Accepted

GitHub evidence: Q5_run1.png and Q5_TM_file.json

ii) Run 2: aaaa#bb - <https://www.automataverse.com/m/R7BLRYbV>



Status: Rejected

GitHub evidence: Q5_run2.png and Q5_TM_file.json

Task D:

This language cannot be recognized in just one left-to-right pass because the machine has to compare the number of as on the left with the number of bs on the right. While reading the as, it does not know yet how many bs are on the other side of the #. So, the machine has to mark the a it is matching with X, move to the right side, and find an unmarked b to match it. After marking the b with Y, it has to move all the way back to the left side and find the next unmarked a. This keeps repeating until all the as have been matched. The machine has to move back and forth because it only has one tape head. It cannot just remember all the as it saw while continuing to the right, so marking them and going back to check the next one makes the comparison possible. My personalized B is 100111. It contains four 1 bits and two 0 bits. For the unary conversion, I used one a for each 1 bit, giving aaaa. I then chose the right-hand side of # according to the required value of j.

Question 6:

Task A:

Input	Current State	R	W	Move	Next State
□	q-even (start)	□	□	R	q-odd
□	q-odd	□	□	R	q-reject
0↓0↓ The input " " or □ is rejected. □□□□□					
Input	Current State	R	W	Move	Next State
1	q-even (start)	1	1	R	q-odd
1	q-odd	1	1	R	q-even
□	q-even	□	□	R	q-odd
□	q-odd	□	□	R	q-reject
0↓0↓0↓0↓ Input "11" is rejected. □11□□□					

GitHub evidence: Q6_TaskA.png

Task B:

The incorrect row in the transition table is:

Current State	Read	Write	Move	Next State
q_even (start)	□	□	R	q_odd

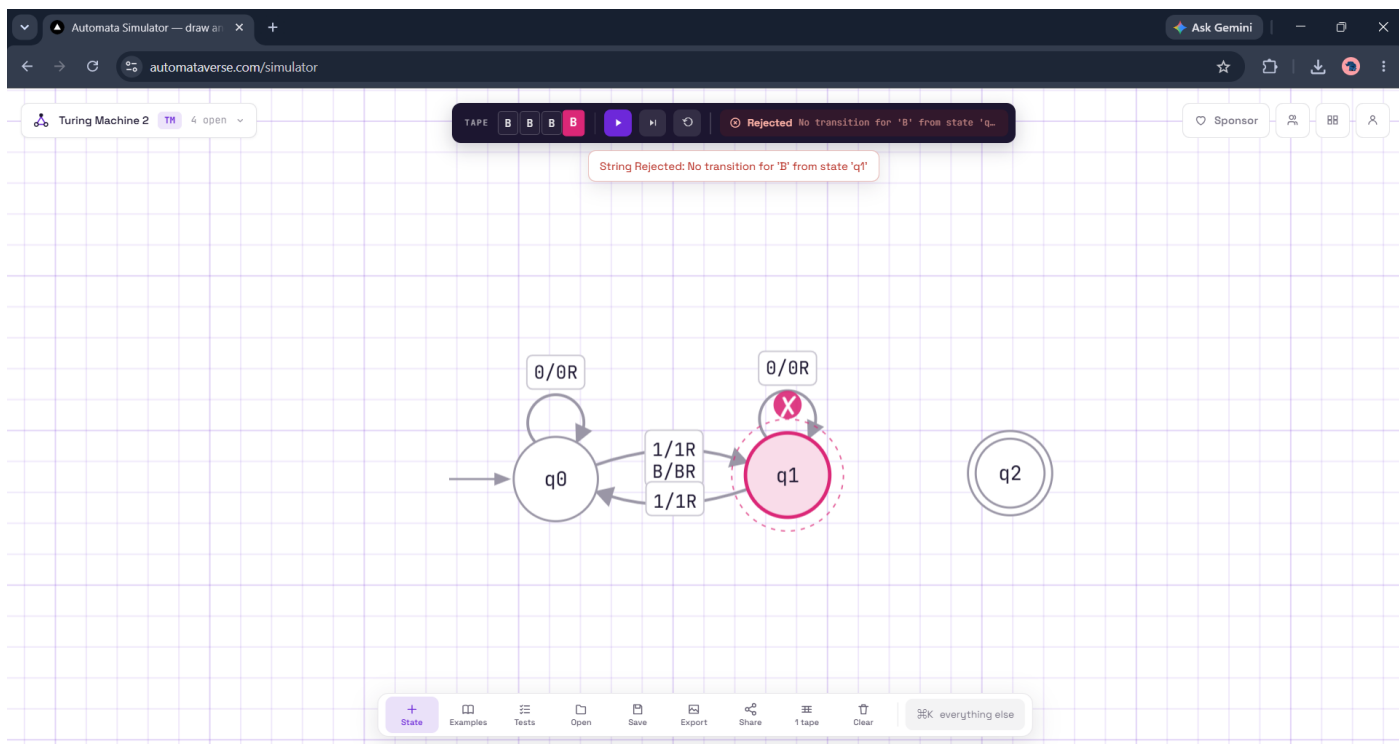
This is because once an input reaches a blank, this means the input's last bit has been read. Since we are in the q_even state as the binary sequence ends, we should move to the accept state, but we don't, leading to all inputs with an even number of 1s getting rejected. The corrected transition should be:

Current State	Read	Write	Move	Next State
q_even (start)	□	□	R	q_accept

Task C:

1. Broken Machine:

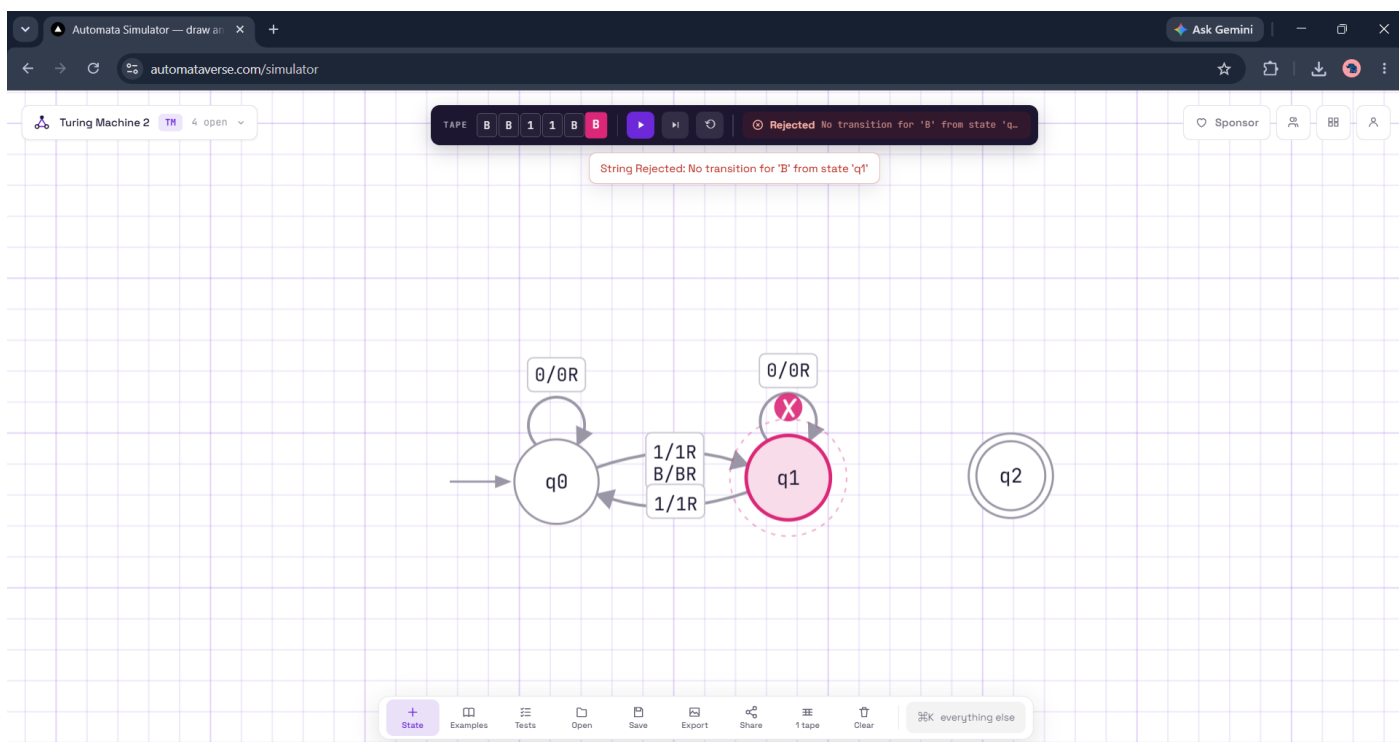
Run 1: " " – <https://www.automataverse.com/m/fCVDGSxi>



Status: Rejected

GitHub evidence: Q6_B_run1.png and Q6_broken_TM_file.json

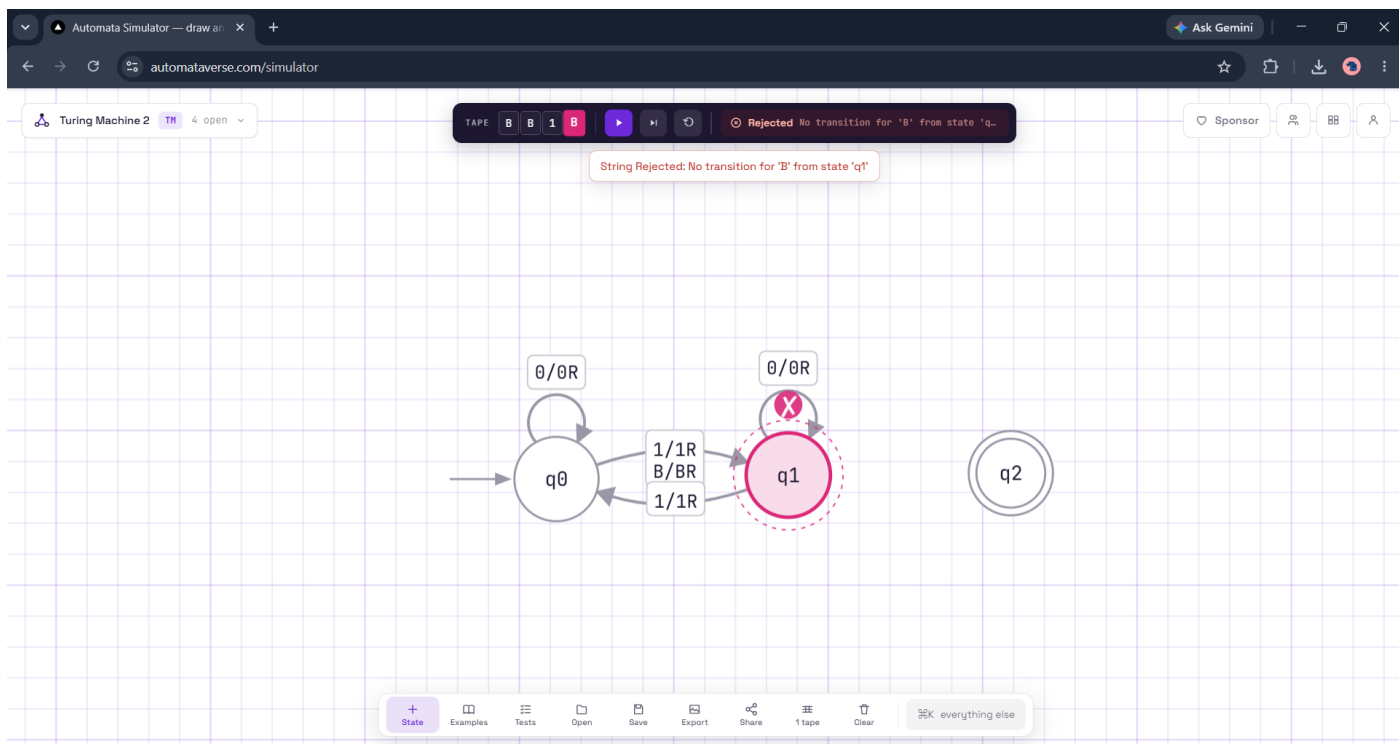
Run 2: 11 – <https://www.automataverse.com/m/fCVDGSxi>



Status: Rejected

GitHub evidence: Q6_B_run2.png and Q6_broken_TM_file.json

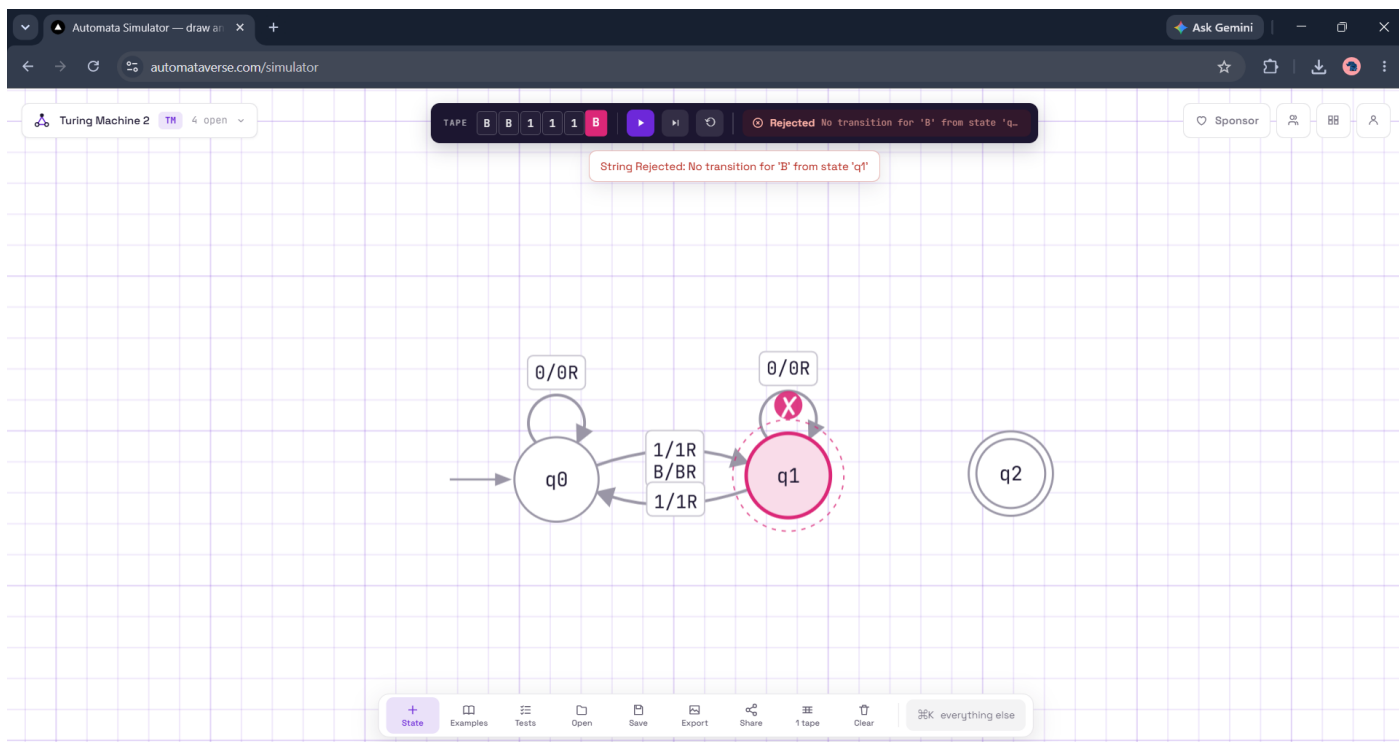
Run 3: 1 – <https://www.automataverse.com/m/fCVDGSxi>



Status: Rejected

GitHub evidence: Q6_B_run3.png and Q6_broken_TM_file.json

Run 4: 111 – <https://www.automataverse.com/m/fCVDGSxi>

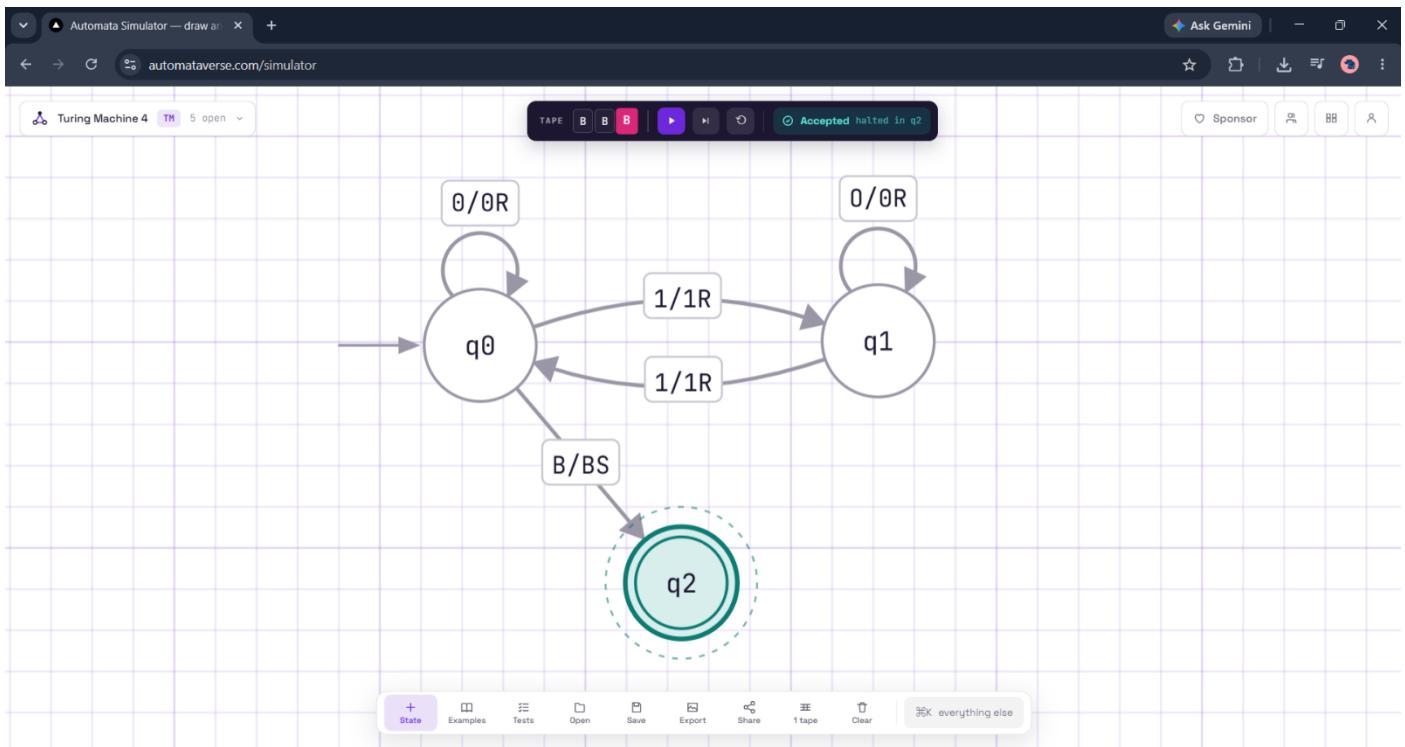


Status: Rejected

GitHub evidence: Q6_B_run4.png and Q6_broken_TM_file.json

2. Corrected Machine:

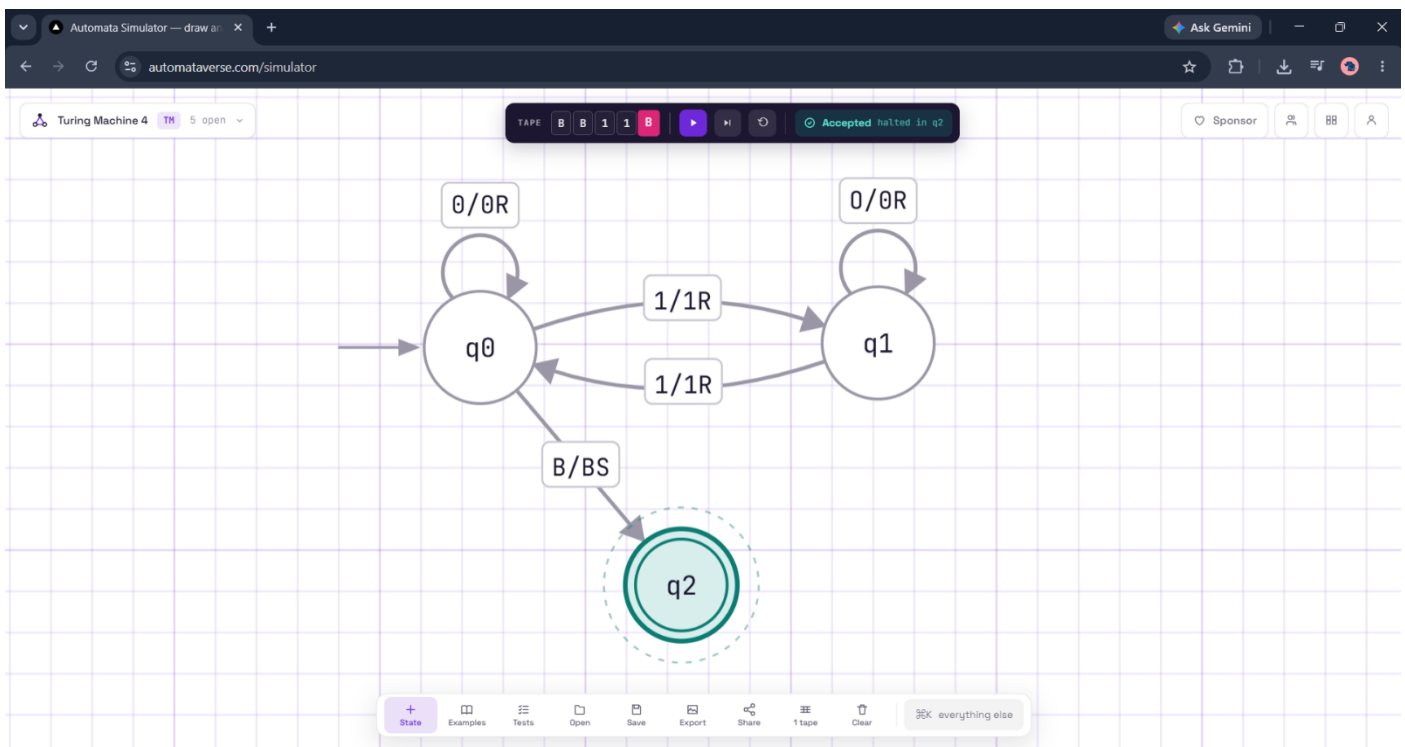
Run 1: " " – <https://www.automataverse.com/m/KRS6EX3U>



Status: Accepted

GitHub evidence: Q6_C_run1.png and Q6_CORRECT_TM_file.json

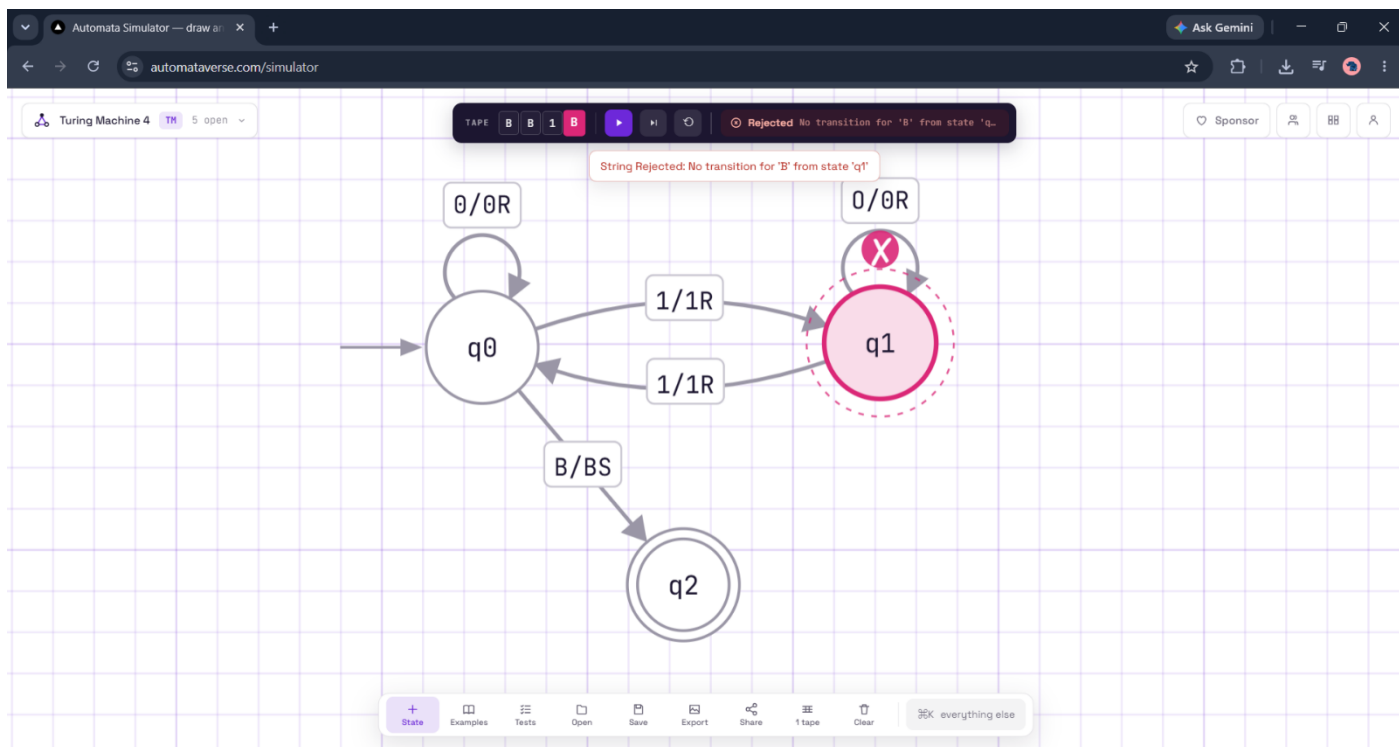
Run 2: 11 – <https://www.automataverse.com/m/KRS6EX3U>



Status: Accepted

GitHub evidence: Q6_C_run2.png and Q6_CORRECT_TM_file.json

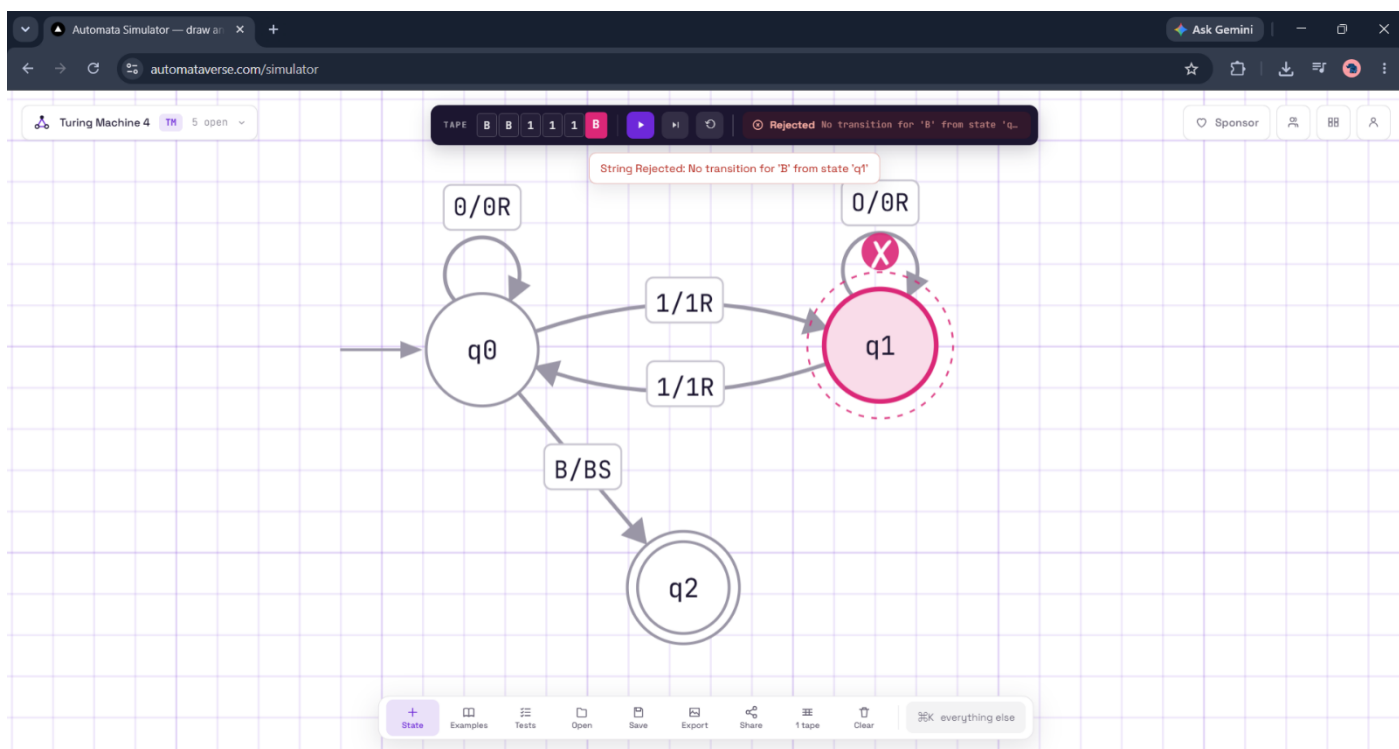
Run 3: 1 – <https://www.automataverse.com/m/KRS6EX3U>



Status: Rejected

GitHub evidence: Q6_C_run3.png and Q6_CORRECT_TM_file.json

Run 4: 111 – <https://www.automataverse.com/m/KRS6EX3U>



Status: Rejected

GitHub evidence: Q6_C_run4.png and Q6_CORRECT_TM_file.json

Input	Correct Status	Broken Machine	Correct Machine
" "	Accept	Reject	Accept
11	Accept	Reject	Accept
1	Reject	Reject	Reject

111	Reject	Reject	Reject
-----	--------	--------	--------

Task D:

If my test value had only been 111 or 1, then I wouldn't have found the error because I would expect the machine to reject them as they have an odd number of 1s, and the broken machine also rejects them. What this tells me is that I should always enter test values that lead to every possible outcome and look for any anomalies or anything that might need a different special case.